

Technical University of Denmark



Course 22168 - Advanced Unix and Python for Bioinformaticians

Resistance to Antibiotics

Identification of antibiotic-resistance genes in metagenomic NGS data using a k-mer based approach

Final Project Report

Academic year: 2025 / 26
Programming language: Python

Authors:
Catarina Anastácio, s253709
Lanjing Buisan Fernandez, s256855

https://github.com/CatarinaAnastacio/Pyhton_Project_Resistance

Table of Contents

Table of Contents.....	2
1. Introduction.....	4
2. Contribution	5
3. Theory: Biological Background.....	6
3.1. DNA and sequencing foundations	6
3.2. FASTA and FASTQ file formats	6
3.3. K-mers and sequence matching.....	6
3.4. Coverage and depth.....	7
3.5. Biological significance and SNP differentiation	7
4. Algorithm Design.....	8
4.1. Algorithm core	8
5. Program Design.....	9
5.1. Program structure	9
5.2. FASTA processing	10
5.3. DNA sequence module	10
5.3.1. Reverse-complement generation	10
5.3.2. K-mer extraction	10
5.4. K-mer index construction.....	10
5.5. FASTQ processing.....	11
5.6. Read-matching algorithm	11
5.7. Coverage, depth and homolog resolution	12
5.7.1. Coverage and depth filtering	12
5.7.2. Homolog collapsing.....	12
5.8. Program design decisions	12
5.9. Design limitations and possible improvements.....	13
6. Program Manual	14
6.1. Running the program	14
6.2. What the program does.....	14
6.3. Output.....	14
7. Results and Analysis	15
8. Runtime Analysis.....	18
8.1. K-mer index construction.....	18
8.2. Read matching	18

8.3. Coverage / depth and homolog resolution.....	18
8.4. Why this approach is efficient	18
8.5. Measured runtime	19
9. Unit Testing	20
9.1. Tests per class	20
9.2. What unit testing revealed	21
10. Conclusion	22
10.1. Strengths	22
10.2. Weaknesses	22
10.3. Future improvements	22
11. References	23

1. Introduction

Antibiotic resistance has become a major worldwide problem, and determining the effectiveness of antibiotics against bacteria is therefore highly relevant in clinical, veterinary and environmental studies. Thus, identifying which resistance genes are present in a sample is essential for the selection of effective treatments or antibiotics.

However, identifying these resistance traits in metagenomic samples, such as stool, blood or pig faeces, can be challenging because the sample contains a mixture of many different bacterial species. Therefore, the aim of this project is to detect bacterial resistance genes that might be present in mixed samples by analysing next-generation sequencing (NGS) data.

The main programming tool used for performing the analysis is Python. The bacterial resistance gene reference is provided as a FASTA file and the metagenomic reads from mixed samples as gzipped FASTQ files. All sequences from both files are processed in 19-nucleotide k-mers, allowing fast comparison between the databases without performing full sequence alignment, which would be more time-consuming. The pipeline reports the resistance genes whose k-mers are consistently and abundantly observed in the sample.

2. Contribution

Both the programming and writing were carried out equally by both authors of the project. For the programming section, GitHub was primarily used to share code, discuss algorithm ideas and evaluate performance. Thereafter, the report was written collaboratively in Google Docs, where both authors edited each other's work.

For both programming and writing, a rotational approach was followed where each section was initially developed by one author and then reviewed and corrected by the other, with roles switching for each new section.

3. Theory: Biological Background

3.1. DNA and sequencing foundations

DNA contains the genetic information required for the development and function of all living organisms. It is made up of nucleotides, each containing one of four bases: adenine (A), cytosine (C), guanine (G), and thymine (T). The order of these bases forms the biological instructions that determine how organisms function.

To study these genetic sequences and identify antimicrobial resistance genes, Next-Generation Sequencing (NGS) is largely used. Unlike traditional sequencing methods, NGS can sequence millions of DNA fragments simultaneously through repeated cycles of nucleotide incorporation and signal detection, making it possible to analyse complex biological samples fast and efficiently.

However, NGS does not produce complete gene sequences directly. Instead, it generates millions of short DNA fragments, called reads, typically between 50–250 base pairs long. These short reads must later be compared and mapped to known reference genes in order to determine which genes are present in the sample. In addition, sequencing technologies introduce small errors and noise, meaning that some reads may contain incorrect nucleotides that are not actually present in the biological material.

3.2. FASTA and FASTQ file formats

Sequencing data from NGS is commonly stored in FASTQ files. A FASTQ file contains both the nucleotide sequence of each read and a quality score for every base. The quality scores are usually represented using the Phred Quality Score system, where a higher score indicates a lower probability of sequencing error.

The resistance reference genes used in this project are stored in FASTA files. FASTA files are simpler than FASTQ files and mainly contain a header identifying the gene followed by the nucleotide sequence itself. In this project, the reads from FASTQ samples are compared against the resistance genes stored in FASTA format.

3.3. K-mers and sequence matching

NGS datasets contain millions of short reads, so comparing every read using full sequence alignment would be computationally unsuitable. Instead, a k-mer based approach is used. A k-mer is a short substring of length k extracted from a DNA sequence where, for a sequence of length L , the total number of k-mers is calculated by $L - k + 1$.

The reference genes from the FASTA database are divided into k-mers and stored in a hash table, allowing very fast lookup of matching sequences. This makes it possible to compare large numbers of sequencing reads efficiently.

The chosen value of k is very important as it directly affects the trade-off between finding true matches and avoiding random false matches (sensitivity vs specificity). Smaller k -mers increase the chance of finding true matches but also increase the number of random matches that are not biologically correct. On the other hand, larger k -mers reduce these false positives but they become more sensitive to sequencing errors. In this project, 19-nucleotide k -mers were chosen, representing a good balance between accuracy and robustness as a 19-base sequence is very improbable to appear randomly in a genome, while still being short enough to reduce the impact of sequencing errors during the matching process.

Since DNA is double-stranded, sequencing reads can come from either strand. Therefore, both the forward sequence of every reference gene and its reverse complement are taken into account during matching.

3.4. Coverage and depth

To confirm that a resistance gene is genuinely present in the sample, it is standard to evaluate how consistently the gene is represented across the sequencing reads. For this purpose, two statistical filters were used: coverage and read depth.

Coverage measures the percentage of the reference resistance gene that is covered by matching k -mers. To reduce the chance of random or partial matches, a threshold of 95 % coverage is used. A complete 100 % threshold was deliberately avoided because sequencing data can contain gaps, sequencing errors or noise. However, a high overall coverage is still a strong indication that the gene is present.

In addition to coverage, **read depth** is also used, with a minimum average depth of 10 being required. This means that multiple independent reads should support the same region of the gene. By applying this filter it is possible to reduce the influence of sequencing noise and it also increases confidence in the presence of resistance genes.

3.5. Biological significance and SNP differentiation

The existence of closely related resistance genes that differ by only a few nucleotides, known as Single Nucleotide Polymorphisms (SNPs), present a significant challenge in resistance gene detection. Due to these similarities, reads originating from one gene may partially match another highly similar gene.

To distinguish between these genes, the overall coverage profile across each gene is evaluated. The gene that is truly present in the sample is expected to show consistent coverage across the entire sequence. In contrast, similar but absent genes will often contain gaps or visible drops in coverage at positions where SNP differences occur.

The final identification therefore prioritises the gene with the strongest and most consistent coverage pattern, helping reduce ambiguity between highly similar resistance genes.

4. Algorithm Design

4.1. Algorithm core

The algorithm consists of different functions that systematically identify which genes are present in the mixed bacterial samples. Its core method is to fragment both the reference resistance genes and the sequencing into k-mers of 19 nucleotides long and compare them against each other.

When matches are detected, further analyses are required to evaluate whether these matches provide valid evidence of gene presence. Individual k-mer matches may happen by chance or as a consequence of sequencing errors, which can result in false positives. Therefore, the algorithm examines the origin of matching k-mers, especially if they come from the same sequencing read and whether they are unique.

If the read sequence truly corresponds to the resistance gene, multiple read k-mers from that read are expected to map to nearby positions among the same gene, forming a consistent alignment pattern. Yet, random matches are normally scattered across the gene and do not exhibit a uniform alignment pattern. In addition, unique k-mers were also taken into consideration as a strict proof of gene identification, as they provide strong and specific-gene evidence.

This positional evidence strongly supports the presence of the gene rather than isolated matches, and helps to differentiate true signals from random matches. Thus, those reads that yield multiple uniform matches alongside the gene are taken into account as strong proof of the gene presence. This strategy provides more specificity and reliability for identifying true matches.

After filtering the matching kmers, coverage and depth are calculated to recognize the relevant resistance genes. The selected genes must have a minimum coverage of 95% and an average depth no less than 10 to be considered as presented resistance genes.

Once filtering is complete, a selection of the 'winner gene' within gene families is performed. The winner gene is defined as the gene exhibiting the highest coverage and depth and whenever present, the strongest proved from unique k-mers. This is a crucial analysis since highly similar genes share most of their sequencing reads and may differ by one or two SNPs. Therefore, these unique k-mer matches help to resolve the ambiguity between homologous genes, enabling the correct identification of the resistance gene presented in the mixed bacteria sample

5. Program Design

The program was designed as a Python pipeline for the identification of antibiotic resistance genes in metagenomic NGS data using a k-mer-based alignment approach. The design focuses on efficiency, scalability and discrimination between highly similar resistance genes. Standard Python libraries (primarily gzip for handling compressed FASTQ files) are used, and no external alignment software is required.

The pipeline is structured into five main components: FASTA parsing, DNA sequence processing, k-mer indexing, FASTQ processing, and statistical analysis and filtering. This separation improves readability, makes the code easier to maintain, and lets each component handle its own task in the pipeline.

5.1. Program structure

The program uses object-oriented programming, where each class is responsible for one specific step of the pipeline. This approach improves organization and its structure is represented in the following pseudocode:

```

Load FASTA reference database
    Store gene sequences and gene lengths

Generate 19-mer index:
    For each gene:
        Extract all 19-mers from the forward strand
        Extract all 19-mers from the reverse-complement strand
        Store each k-mer in a hash table

For each FASTQ read:
    Pre-filter to skip reads with no candidate k-mer
    Extract all read k-mers
    Look them up in the reference index
    Group hits by (gene, alignment shift, strand)
    Identify the best alignment for the read
    Update depth per position for the matching gene (based on kmer matched pos)

For each gene:
    Compute coverage, average depth, minimum depth
    Filter genes below the thresholds

Resolve homologs (winner-takes-all)
Sort and output the final list of resistance genes

```

5.2. FASTA processing

The Fasta class is responsible for reading the resistance gene database stored in FASTA format. It computes two main tasks: extracting the gene headers and nucleotide sequences, and storing both gene lengths and sequences in dictionaries:

- `gene_length[gene_name] = sequence_length`
- `gene_sequences[gene_name] = DNA_sequence`

This structure allows sequence information to be accessed easily in later stages without reprocessing the data from the FASTA file. In order to improve memory efficiency, this processing is implemented as a Python generator (yield), which allows sequences to be processed gradually and avoids loading the entire database into memory, important for large reference datasets.

5.3. DNA sequence module

The DNA module includes all functions that only handle nucleotide sequences, such as reverse-complement generation and 19-mer extraction with positional information.

5.3.1. Reverse-complement generation

Sequencing reads can originate from either DNA strand, so both the forward and reverse-complement orientations of every resistance gene has to be searched. The reverse complement is generated by reversing the DNA sequence and then replacing each nucleotide with its complementary base (A ↔ T and G ↔ C). Any unknown nucleotide is replaced with 'N', which is later filtered out during k-mer generation.

5.3.2. K-mer extraction

The program uses a k-mer length of 19, defined for its optimal balance between sensitivity and specificity. A k-mer is a subsequence extracted from a DNA sequence:

$$k\text{-mer} = \text{DNA}[i : i + 19]$$

The `nineteen_pos()` function extracts overlapping 19-mers along with their exact positions in the gene sequence. For example, for the sequence ACTGATCGATCG..., the first two k-mers are ACTGATCGATCG... (position 0) and CTGATCGATCG... (position 1). Any k-mer containing an ambiguous nucleotide (N) is removed, since their match would be unreliable.

5.4. K-mer index construction

The Kmer class builds the first lookup data structure of the program. All overlapping 19-mers from the resistance genes, and from their reverse complements, are saved in a hash table:

```
kmer_index[kmer] = [(gene_name1, position1, strand1), [(gene_name2,
position2, strand2), ...]
```

This approach is much faster than performing sequence alignments for each read, which allows a more efficient processing of big databases containing millions of sequencing reads when needed. Including both forward and reverse strands ensures that the detection of genes is independent of read orientation.

5.5. FASTQ processing

The `Fastq` class processes compressed sequencing files in FASTQ format. As described earlier, each FASTQ record consists of a read identifier, the DNA sequence, a separator line and a quality scores string. Because this program does not use the quality scores, only the nucleotide sequences are extracted. The compressed `.gz` files are read using Python's `gzip` library and the sequencing reads are processed sequentially using a generator, which avoids the loading of the entire dataset into memory at once.

5.6. Read-matching

The `Match` class contains the main algorithm of the program. It performs matching between sequencing reads and resistance genes using shared k-mers.

First, a pre-filter is applied to every read. Only a subset of its k-mers is checked initially, sampling one k-mer every 20 bases:

```
for pos in range(0, read_len - 18, 20)
```

If none of these sampled k-mers is present in the reference index, the read is discarded. This optimisation reduces unnecessary computation, since most metagenomic reads do not map to any resistance gene.

For reads that pass the pre-filter, every k-mer is then extracted and searched against the reference index. The matching genes and positions are extracted, and the alignment shift between read and gene is computed as:

```
alignment_shift = gene_position - read_position
```

All k-mer matches that share the same gene, alignment shift and strand are grouped together, since they correspond to a single local alignment of the read on the gene. The (gene, shift, strand) group with the largest number of matching k-mers is considered the best mapping for that read. Reads which have the best alignment with fewer than three k-mer matches are discarded because they are likely noise.

After the read is assigned to a gene, the depth value for every nucleotide position that is covered by a matched k-mer is incremented by one:

```
self.dict_depth[gene][p] += 1
```

This produces a depth per position profile across the length of every gene.

5.7. Coverage, depth and homolog resolution

5.7.1. Coverage and depth filtering

For each gene, two statistical measures are calculated using the sequencing depth across all positions. First, gene coverage is calculated using the following formula:

$$\text{Coverage} = (\text{covered_positions} / \text{total_positions}) \times 100$$

Then, the average depth, which represents the number of reads supporting each nucleotide position is calculated as:

$$\text{Average depth} = \text{sum of positional depths} / \text{number of positions}$$

To avoid edge artefacts caused by the natural drop in coverage at the ends of every gene (the first and last 18 bases cannot be fully covered by 19-mers), only the core region of the gene is analysed:

```
core_start = 18
core_end = gene_len - 18
core = pos_counts[core_start:core_end]
```

After computing the statistics, two thresholds are applied to filter out weak or unreliable matches: coverage $\geq 95\%$ and average depth ≥ 10 . Genes that do not meet these criteria are excluded from the final output.

5.7.2. Homolog collapsing

As mentioned before, the presence of resistance genes that share highly similar nucleotide sequences is a significant problem in this project. These genes produce ambiguous matches where reads align equally well to several variants. To resolve this, the program compares k-mers between genes using:

$$\text{similarity} = \text{shared_kmers} / \text{length_smaller_kmer_set}$$

If two genes share similarity above the threshold, only the strongest is kept. To do so, the genes are prioritised by highest coverage, then highest depth, then highest number of unique k-mer matches. This strategy avoids the detection of homologs almost identical and improves biological accuracy.

5.8. Program design decisions

Several design decisions were made to improve performance. Firstly, python dictionaries are used for the k-mer index because they provide fast and efficient, which is essential for the matching step. At the same time, both DNA strands are included to ensure that resistance genes can be detected independently of the orientation of the sequencing reads.

FASTA and FASTQ files are processed gradually using generators instead of being loaded into memory, which reduces memory consumption. An additional optimization is the early filtering of reads, which removes reads with no initial matching k-mers before performing the full analysis. This approach is possible because only the core region of every gene is analysed, so initial k-mers must necessarily be present for a read to contribute.

The program also tracks sequencing depth at every position of every gene, which improves confidence in the detection of resistance genes. Additionally, to better differentiate between highly similar genes, the number of unique k-mer matches per gene is also counted. Finally, using coverage and depth thresholds together with grouping similar genes helps reduce false positives and makes the final results easier to understand.

Overall, these design decisions allow the program to process large datasets efficiently while maintaining good biological accuracy.

5.9. Design limitations and possible improvements

Although the current code implementation is effective, some limitations and approximations should be addressed.

First, the code uses a fixed k-mer length ($k = 19$), chosen as it is the optimal compromise between sensitivity and specificity, as mentioned previously. Although this length performs well overall, there is a possibility that certain datasets might benefit from a different k . Nevertheless, making k configurable would be relatively easy to implement in future versions of the program.

Another limitation is that the algorithm only handles exact k-mer matches. As a result, insertions, deletions and highly altered regions are not represented, which can reduce detection sensitivity for some resistance genes.

A possible future improvement is to integrate the FASTQ quality scores as an additional filter for bases with low quality, which could further reduce noise. Another aspect is memory usage. Although generators are already used while reading the input files, the full k-mer dictionary is still kept in memory. For larger reference databases this could become problematic and would require switching to an improved data structure.

6. Program Manual

The program is executed as a Python script that takes as input one FASTA file with the antibiotic-resistance reference genes, along with one or more gzipped FASTQ files containing metagenomic sequencing reads.

The input files should be in the right format for the pipeline to work properly. The FASTA file should contain headers introduced by '>' followed by nucleotide sequences, and the FASTQ files (.gz) should contain the standard four-line format. The compressed files are read directly using Python's gzip library.

6.1. Running the program

The input files are defined at the bottom of `Project_final_resistance.py`. To run the pipeline:

```
python main.py
```

6.2. What the program does

1. Loads the resistance gene database (FASTA).
2. Builds a 19-mer index for both forward and reverse-complement strands.
3. Reads the compressed FASTQ files and extracts the sequencing reads.
4. Matches read k-mers against the reference index.
5. Maps read depth across positions for each gene.
6. Calculates coverage and average depth for every gene.
7. Filters genes below the coverage / depth thresholds.
8. Removes highly similar homologous genes.
9. Outputs the final list of detected resistance genes.

6.3. Output

The program prints a list of detected resistance genes that passed the coverage ≥ 95 % and average depth ≥ 10 filters together with their gene name, coverage percentage, average depth and minimum depth. Genes are sorted by coverage, then by average depth, then by minimum depth, so the most reliable hits appear first.

```
Resistance genes present in the metagenomic samples:
fosA_3_ACW001000079 | coverage: 100.0 | depth: 1982.9 | min_depth: 490
catB4_1_EU935739 | coverage: 100.0 | depth: 1648.83 | min_depth: 596
blaSHV-28_1_HM751101 | coverage: 100.0 | depth: 1165.98 | min_depth: 729
tet(A)_4_AJ517790 | coverage: 100.0 | depth: 944.75 | min_depth: 591
aac(6')Ib-cr_1_DQ303918 | coverage: 100.0 | depth: 831.63 | min_depth: 607
aac(3)-IIa_1_CP023555.1 | coverage: 100.0 | depth: 799.83 | min_depth: 393
strB_1_M96392 | coverage: 100.0 | depth: 787.78 | min_depth: 549
blaOXA-1_1_J02967 | coverage: 100.0 | depth: 753.12 | min_depth: 555
blaTEM-1B_1_JF910132 | coverage: 100.0 | depth: 752.94 | min_depth: 556
blaCTX-M-15_23_DQ302097 | coverage: 100.0 | depth: 741.78 | min_depth: 436
strA_4_AF321551 | coverage: 100.0 | depth: 734.07 | min_depth: 549
sul2_2_G0421466 | coverage: 100.0 | depth: 632.37 | min_depth: 323
dfrA14_1_DQ388123 | coverage: 99.55 | depth: 717.02 | min_depth: 0
oqxA_1_EU370913 | coverage: 98.6 | depth: 1571.86 | min_depth: 0
oqx8_1_EU370913 | coverage: 96.6 | depth: 1445.25 | min_depth: 0
```

Fig. x. Screenshot of the output, containing gene name, coverage percentage, average depth and minimum depth.

7. Results and Analysis

The pipeline identified multiple antibiotic-resistance genes across the metagenomic sequencing samples. The detected genes show consistently high coverage and strong depth values, indicating reliable identification (Table 1).

Table 1. Antibiotic resistance genes detected in the samples. Coverage indicates the percentage of each gene covered by sequencing reads. Average depth represents the mean number of reads mapping to each nucleotide position within the gene. Minimum depth is the lowest observed depth across the gene.

Gene	Coverage	Average depth	Minimum depth
fosA_3_ACWO01000079	100 %	1982.90	490
catB4_1_EU935739	100 %	1648.83	596
blaSHV-28_1_HM751101	100 %	1165.98	729
tet(A)_4_AJ517790	100 %	944.75	591
aac(6')Ib-cr_1_DQ303918	100 %	831.63	607
aac(3)-IIa_1_CP023555.1	100 %	799.83	393
strB_1_M96392	100 %	787.78	549
blaOXA-1_1_J02967	100 %	753.12	555
blaTEM-1B_1_JF910132	100 %	752.94	556
blaCTX-M-15_23_DQ302097	100 %	741.78	436
strA_4_AF321551	100 %	734.07	549
sul2_2_GQ421466	100 %	632.37	323
dfrA14_1_DQ388123	99.55 %	717.02	0
oqxA_1_EU370913	98.6 %	1571.86	0
oqxB_1_EU370913	96.6 %	1445.25	0

As observed in Table 1, most identified genes exhibit complete or near-complete coverage across their full length. Average depth values range between 600× and 2000×, indicating that the detected genes are supported by many independent reads and not by random matches.

Among these genes, fosA_3_ACWO01000079, catB4_1_EU935739 and blaSHV-28_1_HM751101 stand out as the strongest candidates, with all three reaching 100 % coverage while also exhibiting very high sequencing depth and no significant gaps. This suggests that these genes are particularly abundant in the samples.

On the other hand, three genes, *dfrA14_1_DQ388123*, *oqxA_1_EU370913* and *oqxB_1_EU370913*, showed slightly lower coverage values of 99.55 %, 98.6 % and 96.6 %, respectively. Despite the reduced coverage they still display high average depth and remain above the established thresholds. The small gaps observed are most likely explained by local sequencing bias or noise in specific regions, rather than by absence of the gene from the sample.

An important aspect of this type of study is the ability to distinguish between highly similar homologs that differ only by a few SNPs. Although these genes share many k-mers, differences in coverage gaps and depth distribution allow effective differentiation. This is illustrated by the depth profile of the detected gene *blaTEM-1B_1_JF910132* against its closest homolog *blaTEM-30_1_AJ437107*.

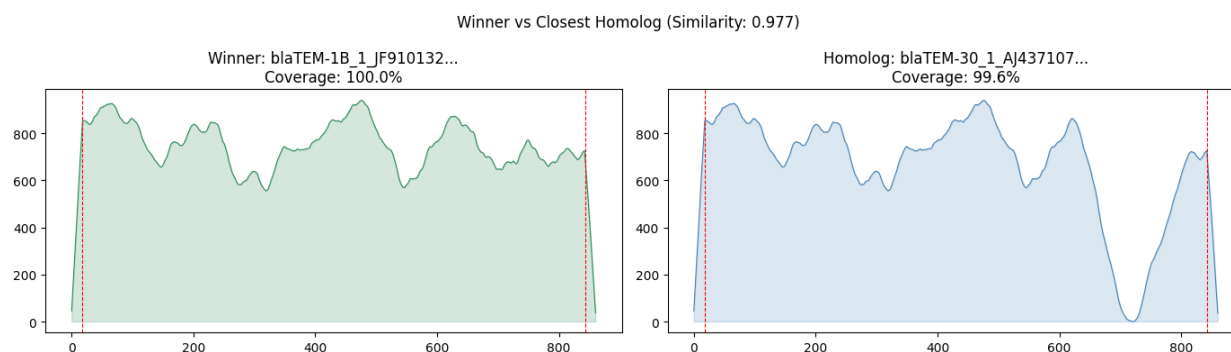


Fig.1. Depth distribution across the detected gene *blaTEM-1B_1_JF910132* and its closest homolog *blaTEM-30_1_AJ437107*. Although both genes share high sequence similarity, coverage gaps in the homolog, likely caused by SNP differences, allow accurate differentiation between the two variants. The full print of statistical features and plots of final genes and their homologs is in GitHub:

https://github.com/CatarinaAnastacio/Pyhton_Project_Resistance

Even though the two genes share around 97.7 % sequence identity, the two genes exhibit distinct mapping profiles. While *blaTEM-1B_1_JF910132* reached full coverage with an average depth of 752.94× and no gaps, the *blaTEM-30_1_AJ437107* homolog showed lower depth and clear coverage gaps, particularly around position 700. These gaps correspond to SNP differences where reads are unable to match the homolog, indicating that its apparent coverage is primarily driven by shared k-mers rather than a true match. Overall, the algorithm correctly identifies the present gene because it maintains read support across the entire sequence, while the homolog loses support at the SNP positions.

Similar patterns were observed for several other homolog and true-gene pairs, including *blaCTX-M-15* vs. *blaCTX-M-3*, the *aac(3)-IIa*, *strA*, *strB* and *tet(A)* variants. In most of these cases the detected gene achieved complete coverage, higher average depth and few or no gaps, while the closest homolog showed reduced coverage and weaker support.

The homolog analysis also exposed limitations of k-mer-based methods. Some genes, such as *blaSHV-28_1_HM751101* and *blaSHV-100_1_AM941846*, produce almost identical coverage patterns because

their sequences are nearly identical in the analysed regions. This suggests that k-mer matching alone may not be sufficient to distinguish between certain variants.

However, genes with extremely high sequence similarity were excluded by the filtering and homolog-identification strategies, which highlights the importance of combining k-mer matching, positional validation, coverage and depth analysis, and unique k-mer evidence. Without all these statistical filters, many homologs could appear as false positives because of shared sequence identity.

Overall, the results show that the pipeline successfully balances sensitivity, specificity, computational efficiency and noise tolerance reasonably well. The combination of high, consistent coverage and sequencing depth strongly indicates that the identified resistance genes are truly present in the samples rather than being random artifacts caused by sequencing noise or sequence similarity.

8. Runtime Analysis

The efficiency of the pipeline was evaluated by breaking down the computational cost into the Big O complexity for each of its primary stages.

8.1. K-mer index construction

The first step in the algorithm consists of building the k-mer index by generating 19-mers from the resistance genes. Since the algorithm iterates through each sequence only once, the complexity for a single gene is:

$O(n)$, where n refers to the length of the resistance gene.

All generated k-mers are then stored in a Python dictionary. This allows for rapid identification because dictionary insertions and lookups have a standard average complexity of **$O(1)$** .

Across the entire database, the total cost of building this index is:

$O(N)$, where N is the total number of bases in the FASTA file

8.2. Read matching

During the matching approach, each sequence read of length m is processed to generate all possible overlapping 19-mers. Because each nucleotide is analysed once, scanning a single read has a complexity of:

$O(m)$, where m is the length of the sequence read.

Each of these k-mers is looked up in the dictionary to find a match. Since the dataset contains R sequence reads, the total algorithm runtime complexity for the matching step is roughly:

$O(R \times m)$, where R is the total number of reads and m is the average read length.

In practice, a pre-filter discards reads that share no k-mers with the database, which significantly speeds up this process.

8.3. Coverage / depth and homolog resolution

Lastly, the pipeline calculates coverage and depth by scanning the data for each gene, which takes **$O(N)$** time. To resolve any similar genes (homologs), the algorithm compares the sequences that passed the initial filter. This step is very fast in practice because the filtering process usually leaves only a small number of genes to compare.

8.4. Why this approach is efficient

The algorithm seems efficient since it uses k-mer for identification of gene presence instead of performing whole sequencing alignment. Full alignment techniques are much slower due to comparing each

sequencing reads nucleotide per reference genes nucleotide. Conversely, dictionaries provide much faster searches for gene identification.

The program is extra optimized given that k-mer reads which possess sequencing errors and do not match with any reference k-mers are discarded for further analysis. Therefore, the program can rapidly select which k-mer matches are reliable, and only those reads are used to calculate coverage and depth.

8.5. Measured runtime

Runtime was measured using Python's time module on the project samples. The full pipeline runs in approximately 2 minutes 30 seconds, well within the recommended 5-minute for this project. The most time-consuming section is the matching loop, which essentially depends on the total number of sequencing reads and on the volume of the resistance genes database.

9. Unit Testing

Unit testing was applied to all six classes that construct the algorithm. All tests are written using pytest and are in a separate test/ folder, while small test data is in testdata/.

9.1. Tests per class

Fasta

For the Fasta class, tests were carried out to verify that the correct number of genes is read from the FASTA file, that the headers and sequences are well extracted, that gene lengths and sequences are correctly stored in their dictionaries, that both dictionaries share the same keys, and that opening a non-existent FASTA raises an exception.

DNA

The DNA class was tested to corroborate reverse complement generation (from simple inputs, mixed sequences and ambiguous bases (N)) and right k-mer collection. The rest of the tests were validating that sequences shorter than 19 nucleotides did not return k-mers, that k-mer possessing ambiguous nucleotides were omitted, that positions are returned in increasing order, and the total number of k-mers matches $L - 18$.

Kmer

The Kmer class test confirmed that index k-mer was generated correctly, that both forward and reverse strand were taken into account, and that the stored entries had the correct gene name, position and strand data.

Fastq

For the FASTQ class, tests were verifying that sequencing reads were extracted correctly from zip FASTQ files, that only the DNA sequence was yielded (no headers, '+' or quality strings), that empty files were properly managed, and that files containing only non-matching reads are still parsed correctly.

Match

The Match class was tested to validate the right initialization of depth arrays, to properly update depth values after matching the k-mers, and to handle reads that are not long enough or with sequencing errors.

Analysis

Finally, the Analysis class was tested to confirm coverage and depth filtering, excluding genes that did not commit the requirements for filtration, managing short genes and proper election of the 'winner gene' across highly similar genes according to coverage, depth and unique k-mer evidence.

9.2. What unit testing revealed

The unit testing helps to identify and verify possible error scenarios in the program and how to handle them. Specially for k-mer matching, coverage and depth calculations, and unique k-mer filtration. This is where most of the errors could be.

10. Conclusion

Overall, the project succeeded by finding the resistance genes, indicating their coverage percentage and average depth, that were present in the bacterial mixed sample. The program was able to run in less than 5 minutes inputting both FASTQ zip and resistance genes FASTA files.

The main challenge was to determine the appropriate criteria for matching k-mers selection and time optimization. However, both issues were solved effectively. Coverage, depth and unique k-mers were the main basis for selecting the 'winner genes'.

For future improvements, stronger understanding on the biological background, specially Next Generation Sequencing (NGS) principles and biology context should be acquired prior to starting the project. The program could be further complex to reduce time consuming and improve result accuracy.

Finally, the project achieved the main output and provided a solid algorithm for future improvements and enhancements.

11. References

1. Wad-Sackett, P. *Resistance to Antibiotics — project description, course 22118*, DTU Health Tech. Available at: https://teaching.healthtech.dtu.dk/22118/index.php/Resistance_to_antibiotics
2. Hu, T., Chitnis, N., Monos, D., & Dinh, A. (2021). *Next-generation sequencing technologies: An overview.* *Human Immunology*, 82(11), 801–811. <https://doi.org/10.1016/j.humimm.2021.02.012>